Федеральное государственное бюджетное образовательное учреждение
высшего образования

«Тамбовский государственный университет имени Г.Р. Державина»

Институт экономики, информационных технологий и креативных индустрий

Кафедра математического моделирования и информационных технологий

ДИПЛОМНАЯ РАБОТА

на тему

«Разработка программного средства для комплексной оптимизации производительности и конфиденциальности операционной системы Windows»

Студент четвертого курса специальности «Прикладная информатика» очного отделения: Усов И. П.

Научный руководитель: д.п.н., профессор, профессор кафедры ММиИТ Чванова М. С.

Тамбов - 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ОПТИМИЗАЦИИ ОПЕРАЦИОННОЙ СИСТЕМЫ WINDOWS 6
1.1. Архитектура операционной системы Windows и основные механизмы управления системными ресурсами 6
1.2. Понятие производительности операционной системы и методы её повышения 11
1.3. Конфиденциальность в операционной системе Windows: источники телеметрии и риски утечки данных 14
1.4. Обзор и сравнительный анализ существующих программных твикеров для Windows 18
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА ДЛЯ КОМПЛЕКСНОЙ ОПТИМИЗАЦИИ WINDOWS 22
2.1. Постановка задачи и требования к программному средству 22
2.2. Архитектура программного средства и выбор технологий реализации 25
2.3. Классификация и описание реализуемых системных твиков 28
2.4. Механизмы безопасного применения твиков и восстановления состояния системы 32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 36

ВВЕДЕНИЕ

Windows остаётся основной настольной платформой для большинства домашних и учебных компьютеров, офисных рабочих мест и игровых систем. Один и тот же компьютер с Windows может ощущаться по-разному: многое зависит не только от процессора, памяти или накопителя, но и от конфигурации самой ОС. На скорость и удобство влияют активные службы, схема электропитания [7], сетевые параметры, автозагрузка и фоновая диагностическая активность [8].

Пользователь обычно приходит к настройке системы не из теоретического интереса. Его раздражает лишняя активность в фоне, телеметрия, задержки интерфейса, перегруженная автозагрузка или неудобное поведение мыши и клавиатуры. Часть этих проблем решается штатными средствами Windows, но нужные параметры разбросаны по реестру, оснасткам, командной строке, политикам и планировщику задач [6] [7] [9]. Из-за этого даже простая оптимизация превращается в набор несвязанных действий, где легко ошибиться или забыть, что именно было изменено.

Актуальность темы. Разработка программного средства для комплексной оптимизации Windows сохраняет практическую значимость. Пользователю нужен не набор отдельных скриптов или «секретных» параметров, а понятный инструмент [7] [10] для проверки состояния системы, применения выбранных изменений, понимания их смысла и возврата к исходной конфигурации при необходимости. Такие операции должны оставаться воспроизводимыми, иначе настройка быстро превращается в хаотическое вмешательство в работу операционной системы.

Проблема исследования. Существующие твикеры для Windows [3] [4] [5] обычно закрывают только часть пользовательских задач. Одни делают упор на кастомизацию интерфейса, другие — на приватность, третьи — на игровую производительность. При этом такие инструменты не всегда ясно показывают, какие изменения вносятся, какие риски с ними связаны и можно ли без лишних действий вернуть прежнее состояние. Поэтому сохраняется потребность в едином программном средстве, где мониторинг, структурированный набор твиков и безопасное восстановление работают как части одного сценария.

Цель исследования. Цель выпускной квалификационной работы — разработать программное средство для комплексной оптимизации производительности и конфиденциальности операционной системы Windows и оценить эффективность его применения.

Задачи исследования. Для достижения поставленной цели в работе необходимо решить следующие задачи:

1. Проанализировать архитектуру операционной системы Windows и механизмы управления системными ресурсами.
2. Рассмотреть основные подходы к повышению производительности и обеспечению конфиденциальности в Windows.
3. Выполнить обзор и сравнительный анализ существующих программных твикеров для Windows.
4. Сформулировать требования к разрабатываемому программному средству.
5. Спроектировать архитектуру программного средства для комплексной оптимизации системы.
6. Реализовать программное средство с поддержкой набора системных твиков и сервисных инструментов.
7. Провести тестирование и оценить влияние разработанного решения на параметры производительности и конфиденциальности системы.

Объект исследования. Объект исследования — операционная система Windows как программная платформа управления аппаратными и программными ресурсами вычислительной системы.

Предмет исследования. Предмет исследования — методы и программные средства оптимизации производительности и конфиденциальности операционной системы Windows путём изменения её системных параметров и конфигураций.

Гипотезы исследования. В ходе выполнения работы принимаются следующие гипотезы:

1. Комплексное и систематизированное применение твиков способно повысить производительность Windows без заметного ухудшения её стабильности.
2. Автоматизация изменения системных параметров снижает вероятность ошибок по сравнению с ручной настройкой.
3. Наличие механизмов контроля состояния и отката повышает безопасность применения твиков.
4. Ограничение телеметрии и избыточной фоновой активности [6] [9] способствует повышению уровня конфиденциальности пользователя.

Структура работы. Работа включает введение, три главы, заключение, список использованных источников и приложения.

ГЛАВА 1. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ ОПТИМИЗАЦИИ ОПЕРАЦИОННОЙ СИСТЕМЫ WINDOWS

1.1. Архитектура операционной системы Windows и основные механизмы управления системными ресурсами

Операционная система Windows представляет собой сложную программную среду, в которой прикладные программы, системные службы и аппаратные устройства работают не изолированно, а в рамках общей архитектуры. Именно устройство этой архитектуры определяет, какие параметры можно изменять без критических последствий, а какие требуют особенно осторожного подхода. Поэтому вопросы оптимизации лучше начинать не с отдельных твиков, а с базовых принципов работы операционной системы Windows [1] [2] [8].

В основе Windows лежит разделение на пользовательский режим (User Mode) и режим ядра (Kernel Mode). Такое деление нужно не для формальности, а для практической устойчивости системы. Пользовательские приложения работают в ограниченной среде и не получают прямого доступа к критическим ресурсам компьютера. Если программа завершится с ошибкой, в большинстве случаев это затронет только её саму. Компоненты режима ядра, напротив, отвечают за работу с памятью, процессором, драйверами и устройствами, поэтому сбой на этом уровне, как показывают материалы по архитектуре ядра Windows [1] [2] [8], уже способен нарушить работу всей системы.

Пользовательские программы работают с Windows через прикладные интерфейсы: Win32, .NET и компоненты UWP. Запрос сначала попадает в системные библиотеки и только затем, если это требуется, переходит на уровень ядра. Даже обычный запуск приложения или открытие файла проходит через несколько слоёв программной логики. Если эти слои настроены рационально, система отвечает быстрее; эта связь хорошо прослеживается в описаниях архитектуры Windows NT [1] [2] [8].

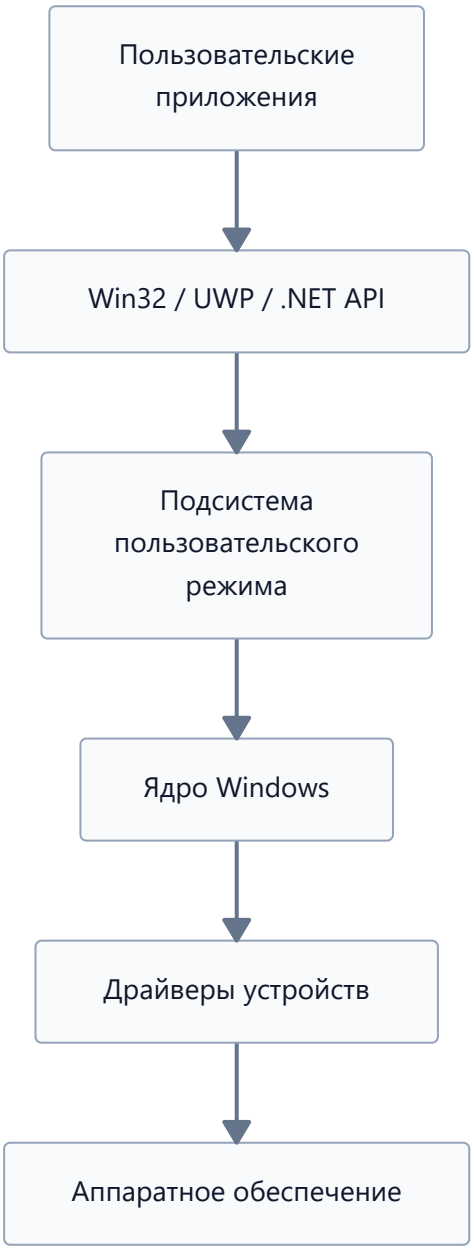

Рисунок 1. Общая архитектура Windows

В режиме ядра Windows можно выделить несколько ключевых подсистем, которые непосредственно связаны с управлением ресурсами:

- планировщик процессов и потоков;
- диспетчер памяти;
- менеджер ввода-вывода;
- менеджер объектов;
- менеджер электропитания.

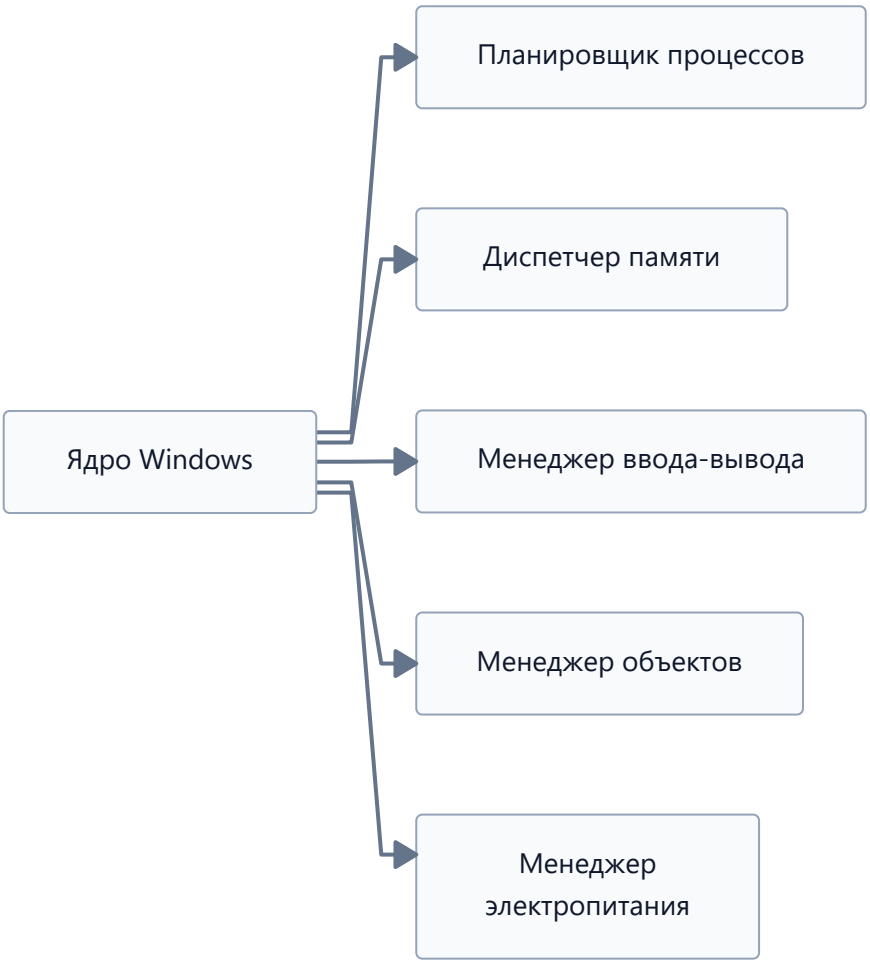

Рисунок 2. Режим ядра и его компоненты

Планировщик процессов и потоков отвечает за распределение процессорного времени. Для пользователя это проявляется в том, насколько быстро система реагирует на действия, как ведут себя программы под нагрузкой и насколько заметны фоновые процессы. Если в фоне работают службы, обновления, телеметрия или тяжёлые приложения из автозагрузки, они конкурируют за ресурсы с задачами, которые пользователь считает приоритетными. Поэтому оптимизация производительности часто касается именно числа фоновых процессов, схем электропитания [7] и параметров планировщика [1].

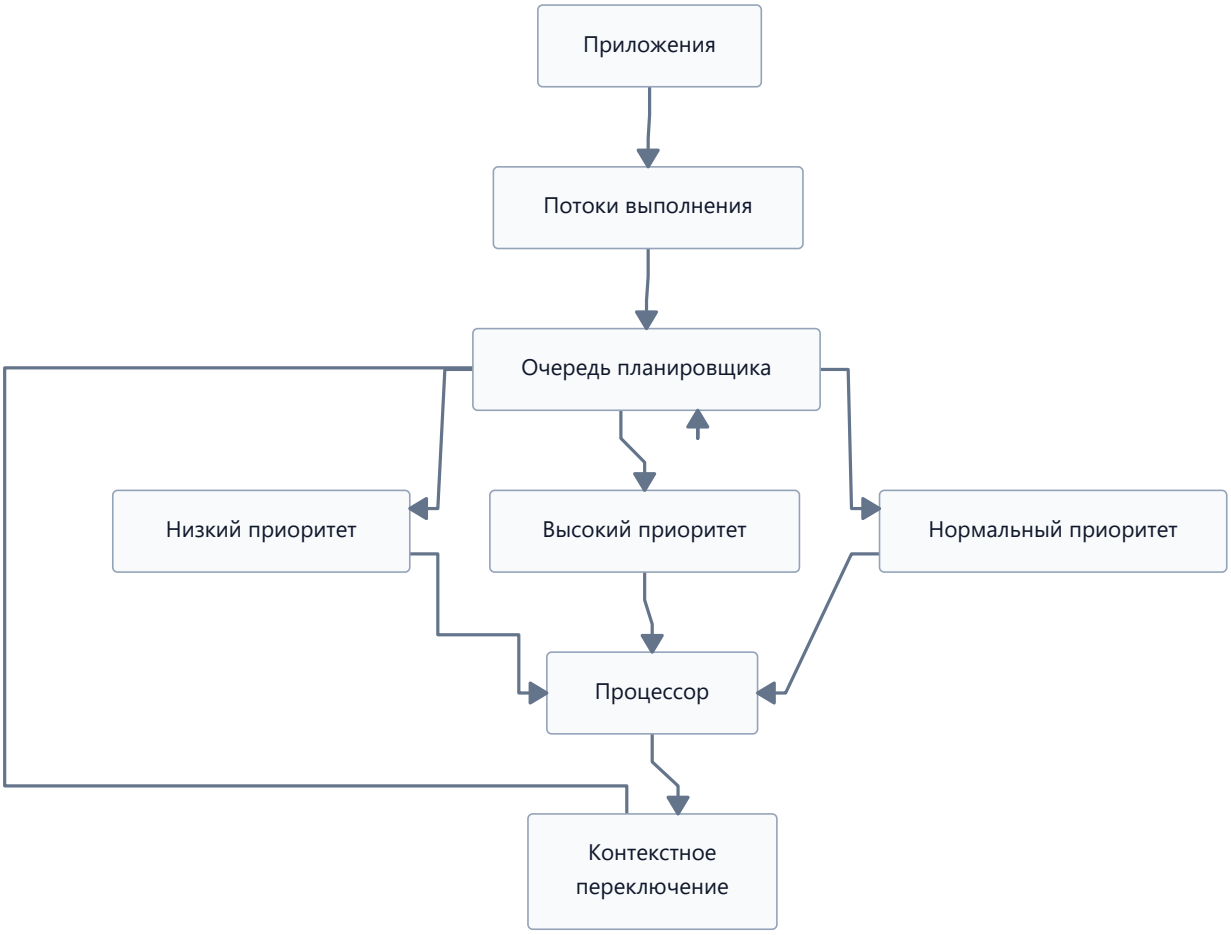

Рисунок 3. Управление процессором

Диспетчер памяти влияет на поведение системы не меньше планировщика. Windows использует виртуальную память: каждый процесс получает собственное адресное пространство, что повышает устойчивость и упрощает многозадачность. Но при нехватке ОЗУ система чаще обращается к файлу подкачки. Тогда исправный с аппаратной точки зрения компьютер начинает работать заметно медленнее: программы дольше переключаются, накопитель получает больше запросов, интерфейс теряет плавность. Поэтому в область оптимизации попадают кэширование, фоновое потребление памяти и поведение служб, связанных с управлением памятью [1] [2] [7].

Рисунок 4. Управление памятью

Отдельного внимания заслуживает подсистема ввода-вывода. Через неё проходят обращения к накопителям, сетевым интерфейсам и периферийным устройствам. На этом уровне производительность системы во многом зависит не только от аппаратной части, но и от драйверов, фоновых служб, настроек сети и накопителей. Например, избыточная фоновая активность Windows [7], неудачная конфигурация драйверов или постоянные диагностические запросы способны увеличивать задержки и создавать ощущение «тяжёлой» системы даже на современном оборудовании.

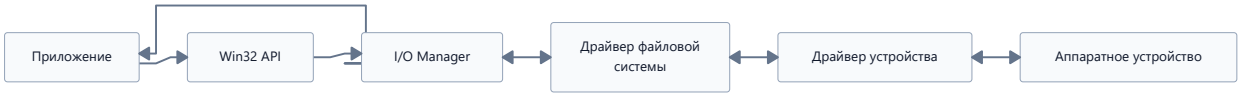

Рисунок 5. Управление устройствами и вводом-выводом

Архитектура Windows напрямую задаёт рамки для любых действий по оптимизации. Твики, которые применяет пользователь или специализированная программа, воздействуют на уже существующие механизмы: режимы питания, конфигурацию служб, параметры обработки памяти, сетевые настройки и отдельные характеристики пользовательского интерфейса. Чем яснее разработчик понимает связь этих механизмов в архитектуре Windows [1] [2] [7], тем выше вероятность получить реальный эффект без ущерба для стабильности системы.

1.2. Понятие производительности операционной системы и методы её повышения

Производительность операционной системы обычно связывают с «скоростью работы компьютера», однако на практике это понятие значительно шире. Для Windows важна не только вычислительная мощность, но и то, насколько быстро запускаются приложения, как система ведёт себя в многозадачном режиме, насколько плавно реагирует интерфейс и как меняется состояние компьютера под длительной нагрузкой. Поэтому производительность разумнее рассматривать как совокупность характеристик, которые показывают эффективность использования аппаратных ресурсов Windows [7] в реальных сценариях работы.

К основным показателям производительности относятся загрузка процессора, использование оперативной памяти, интенсивность ввода-вывода, скорость старта приложений и отзывчивость интерфейса. Проблема не всегда находится в одном месте. На слабом ноутбуке узким местом может быть память, на рабочей станции — фоновые службы, на игровой системе — лишняя сетево-дисковая активность. Поэтому оптимизация Windows не сводится к одной «волшебной» настройке и требует комплексного подхода [7] [10].

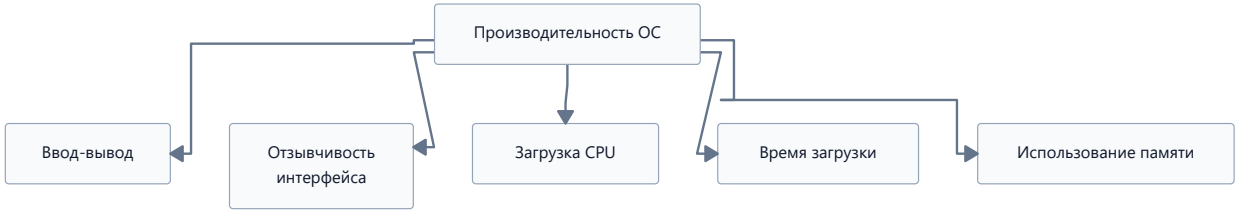

Рисунок 6. Показатели производительности операционной системы

Существенную роль играет процессорная подсистема. В Windows планировщик распределяет время между потоками с учётом приоритета и текущей нагрузки. Если в системе одновременно активно работают обновления, антивирусное сканирование, приложения из автозагрузки и пользовательские программы, доступное процессорное время рассеивается между множеством задач. Поэтому практические методы оптимизации на этом уровне обычно связаны с сокращением числа лишних фоновых процессов, более аккуратной настройкой автозагрузки и корректным выбором схемы электропитания [7].

Рисунок 7. Влияние схемы электропитания

Не менее заметно на производительность влияет работа памяти. При достаточном объёме ОЗУ система может удерживать активные данные в быстром доступе и реже обращаться к накопителю. Когда памяти не хватает, Windows начинает интенсивнее использовать файл подкачки, и это сразу сказывается на практической работе: замедляется переключение между задачами, появляются микрозадержки, возрастает нагрузка на диск. Отсюда вытекает практический вывод: контроль потребления памяти, фоновых приложений и лишней активности служб улучшает поведение системы даже без аппаратного апгрейда [7].

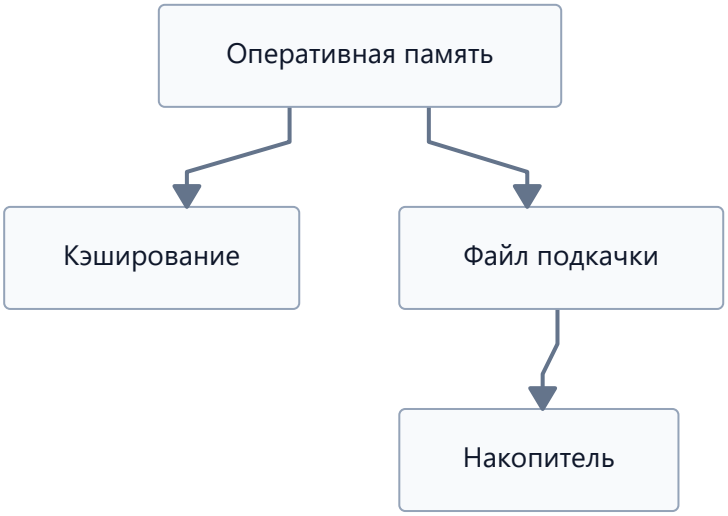

Рисунок 8. Управление оперативной памятью

Подсистема ввода-вывода также часто становится узким местом. Даже при быстром процессоре заметные задержки возникают тогда, когда накопитель перегружен фоновыми задачами, служба индексации постоянно обраща-

ется к файлам, а система одновременно выполняет резервное копирование, обновление или телеметрические операции. С точки зрения пользователя это проявляется в медленном запуске программ, задержках при открытии документов, нестабильной работе игр и росте времени отклика приложений. Поэтому оптимизация I/O в Windows сводится к сокращению лишних фоновых обращений к диску и сети, а также к настройке соответствующих служб и драйверов [7] [10].

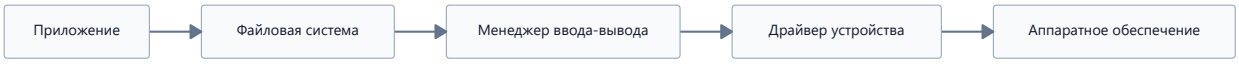

Рисунок 9. Подсистема ввода-вывода

На общую производительность заметно влияют и системные службы. Часть из них необходима для штатной работы Windows, однако существует и такая фоновая активность, которая в конкретном пользовательском сценарии не даёт ощутимой пользы. Речь идёт о телеметрии, диагностических задачах, отдельных облачных функциях, навязчивых автопроверках и сторонних службах, устанавливаемых вместе с прикладными программами. Их влияние не всегда выражается в постоянной высокой нагрузке; гораздо чаще проблема проявляется в кратковременных пиках использования процессора, памяти, диска или сети [7] [10], которые ухудшают субъективную плавность работы.

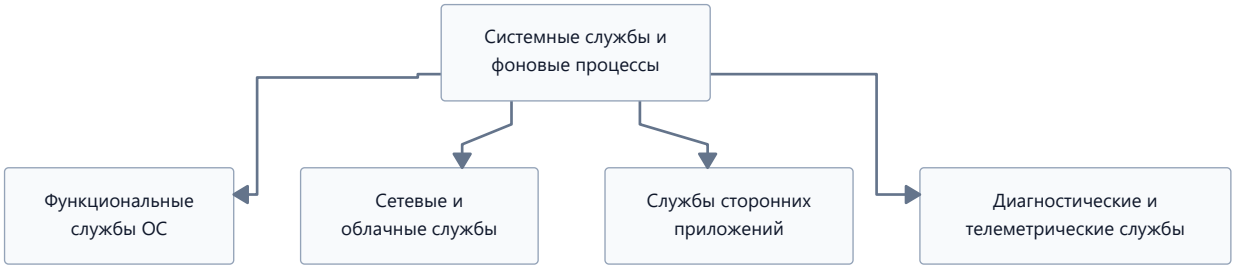

Рисунок 10. Системные службы и фоновые процессы

Отдельного упоминания заслуживает отзывчивость пользовательского интерфейса. Даже если средняя загрузка процессора невелика, пользователь может воспринимать систему как медленную из-за подтормаживаний анимации, задержек ввода, нестабильной работы оконного менеджера или неровной реакции указателя мыши. В Windows на это влияют графическая подсистема, схема электропитания, обработка событий ввода и фоновая активность приложений. Поэтому в практике оптимизации важны не только «синтетические» показатели, но и общее ощущение плавности системы [7].

Повышение производительности Windows требует согласованной настройки нескольких подсистем сразу. Изолированные изменения иногда дают локальный эффект, но устойчивый результат чаще появляется тогда, когда оптимизация затрагивает автозагрузку, фоновые службы, параметры питания, сетевую активность и отдельные особенности пользовательского интерфейса [7] [10]. На этой логике строятся современные программные средства комплексной оптимизации.

1.3. Конфиденциальность в операционной системе Windows: источники телеметрии и риски утечки данных

Вопрос конфиденциальности в Windows в последние годы перестал быть узкоспециальной темой для системных администраторов. Для обычного пользователя он также стал практически значимым, поскольку современные версии Windows активно используют телеметрию, облачную синхронизацию, диагностические службы и связанные с ними фоновые задачи. С инженерной точки зрения у этих механизмов есть понятное назначение: они помогают разработчику выявлять ошибки, анализировать стабильность системы и собирать информацию о типовых сценариях использования. Однако для защиты данных такая ситуация [6] [9] остаётся неоднозначной.

Телеметрия в Windows — это автоматический сбор и передача сведений о работе ОС, оборудования и приложений. В такие сведения могут входить конфигурация устройства, установленные драйверы, частота сбоёв, использование отдельных функций и другие параметры. Для Microsoft такой сбор связан с улучшением продукта. Для пользователя это одновременно означает постоянный канал передачи информации о состоянии компьютера. Чем шире набор собираемых сведений, тем выше риск появления в нём чувствительных данных или косвенных признаков пользовательской активности [6] [9].

Microsoft разделяет диагностические данные на обязательные и дополнительные. Обязательный уровень связан с базовой информацией о системе, безопасностью и критическими сбоями. Дополнительный уровень предполагает более детальное описание поведения системы и приложений. На практике проблема заключается не только в самом факте разделения на уровни, но и в том, что пользователь далеко не всегда понимает, какие именно данные [6] [9] входят в каждый из них и как часто они отправляются.

Источники телеметрической и диагностической активности в Windows удобно разделить на несколько групп. Первая группа — системные службы, в том числе DiagTrack и связанные компоненты для сбора диагностики, push-механизмов и отчётов об ошибках. Вторая — запланированные задачи, которые

запускаются через планировщик и продолжают работать даже тогда, когда пользователь редко открывает соответствующие разделы настроек. Третья группа связана с облачными функциями Windows: синхронизацией параметров, рекламным идентификатором, учётной записью Microsoft и отдельными сервисами облачной аналитики [6] [9] [10].

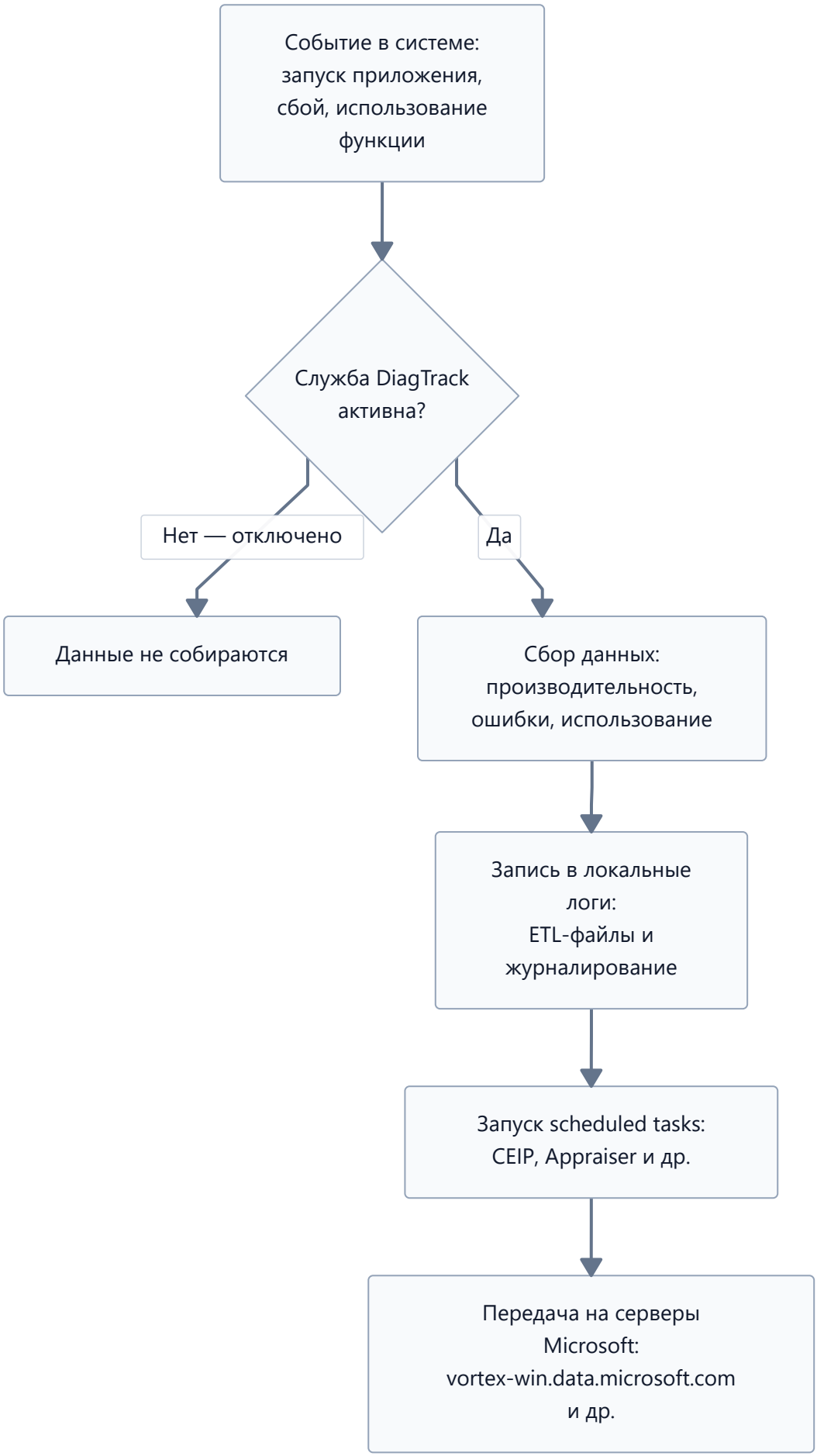
Событие в системе: запуск приложения, сбой, использование функции
Служба DiagTrack активна?
Нет — отключено
Да
Данные не собираются
Сбор данных: производительность, ошибки, использование
Запись в локальные логи: ETL-файлы и журналирование
Запуск scheduled tasks: CEIP, Appraiser и др.
Передача на серверы Microsoft: vortex-win.data.microsoft.com и др.

Рисунок 11. Процесс сбора и передачи телеметрии

С точки зрения конфиденциальности наибольший интерес представляют не столько сами названия служб или задач, сколько типы данных, которые могут передаваться. К ним относятся сведения о конфигурации оборудования, установленном программном обеспечении, параметрах сети, истории использования функций, а при определённых сценариях — и данные из отчётов о сбоях. Последняя группа особенно чувствительна, поскольку в дампах памяти теоретически могут содержаться фрагменты информации, связанные с работой приложений, открытыми документами или пользовательским сеансом [6] [9].

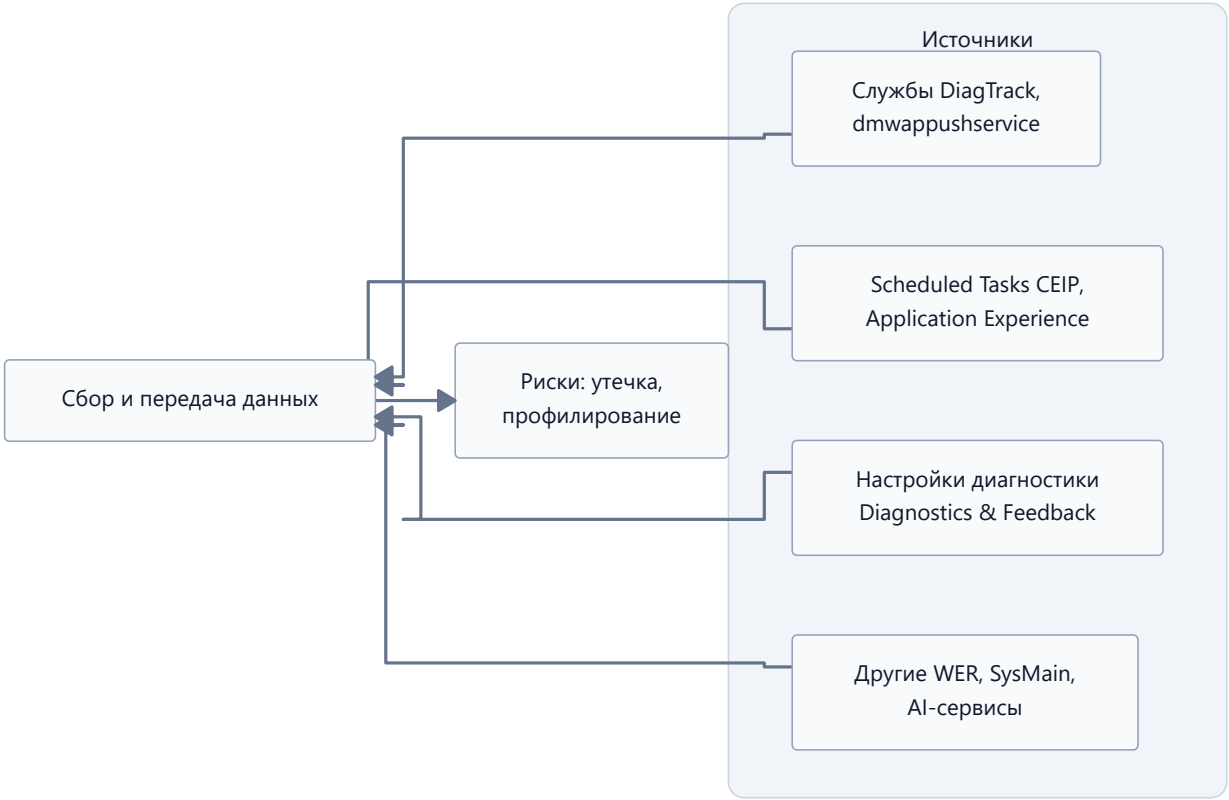

Рисунок 12. Основные источники телеметрии

Риски телеметрии проявляются на нескольких уровнях. Первый — избыточность данных: пользователь не всегда понимает, какой объём информации покидает устройство и зачем он нужен. Второй — профилирование. Даже формально обезличенные сведения при регулярном сборе могут описывать устройство, привычки использования и набор установленного ПО. Третий уровень связан с сетевой безопасностью: каждая дополнительная передача данных расширяет поверхность взаимодействия системы с внешней инфраструктурой. Есть и ресурсный аспект: диагностические службы, отчёты и фоновые задания потребляют процессорное время, память и сетевой трафик [6] [9] [10].

В то же время полный отказ от всех диагностических механизмов нельзя считать универсально правильным решением. В корпоративной среде часть таких данных используется для поддержки, мониторинга парка устройств и выявления проблем совместимости. Поэтому вопрос конфиденциальности в Windows сводится не к отключению всего возможного, а к выбору разумного баланса между функциональностью системы и допустимым уровнем передачи данных [6].

Поэтому в современных твикерах блок настроек конфиденциальности обычно занимает одно из центральных мест. Пользователю нужен не просто набор флагов «вкл./выкл.», а понимание того, какие механизмы затрагиваются: службы, задачи планировщика, рекламные идентификаторы, синхронизация в облаке, сбор рукописного и текстового ввода, отчёты об ошибках и другие каналы. В этом случае оптимизация конфиденциальности становится более осознанной и связывается с архитектурой прикладного инструмента [6] [9] [10], рассматриваемого в последующих главах.

1.4. Обзор и сравнительный анализ существующих программных твикеров для Windows

Интерес к программным твикерам для Windows во многом объясняется тем, что штатный интерфейс операционной системы далеко не всегда даёт быстрый доступ к нужным настройкам. Часть параметров скрыта в реестре, часть распределена между служебными оснастками, а часть связана сразу с несколькими механизмами, например со службой, заданием планировщика и значением политики. Из-за этого пользователи и администраторы обращаются к специализированным утилитам [3] [4] [5], чтобы выполнять такие изменения централизованно и заметно быстрее.

Большинство твикеров работает поверх стандартных механизмов Windows: редактирует реестр, меняет конфигурацию служб, отключает задачи планировщика, корректирует параметры интерфейса или удаляет лишние компоненты. Разница между ними заключается не столько в наборе технических приёмов, сколько в приоритетах. Одни программы ориентированы на кастомизацию интерфейса, другие — на приватность, третьи — на игровую производительность или агрессивное удаление встроенных компонентов Windows [3] [4] [5] [10].

Среди наиболее известных инструментов выделяются Winaero Tweaker, Ultimate Windows Tweaker, Hone и Win10Tweaker. Эти программы выбраны для сравнения потому, что они представляют разные подходы к оптимизации

Windows и наглядно показывают сильные и слабые стороны класса твикеров [3] [4] [5] в целом.

Winaero Tweaker традиционно воспринимается как универсальный инструмент для тонкой настройки интерфейса и поведения Windows. Его сильная сторона — широкий набор параметров, связанных с контекстными меню, внешним видом Проводника, панелью задач и восстановлением старых поведенческих схем. При этом Winaero Tweaker не ограничивается визуальной частью и включает настройки приватности, обновлений и ряда системных параметров. Такой инструмент удобен для опытного пользователя, который хочет точечно изменять поведение системы и хорошо понимает последствия каждого шага [3].

Ultimate Windows Tweaker ориентирован на более структурированный и понятный подход. В нём твики разложены по категориям, связанным с приватностью, безопасностью, производительностью и системными параметрами. Это делает инструмент проще для восприятия и снижает порог входа. Одновременно именно такая модель показывает ограничение большинства существующих твикеров: пользователь получает набор переключателей, но не всегда видит реальное текущее состояние системы, контекст применения изменения и его связь с другими подсистемами [4].

Hone занимает другую нишу. Этот инструмент акцентирует внимание на игровых сценариях, снижении задержек и быстром применении заранее подобранных настроек. Его преимущество заключается в низком пороге использования: многие операции выполняются почти в режиме «одного действия». Однако такая простота имеет обратную сторону. Чем выше степень автоматизации и чем меньше пользователю показывают технический контекст, тем труднее оценить, какие именно изменения были сделаны и как их безопасно откатить [5] в дальнейшем.

Win10Tweaker, напротив, ассоциируется с более агрессивным подходом. Подобные инструменты обычно сочетают богатый набор возможностей с режимами глубокой очистки системы, отключения служб и удаления компонентов. Это делает их привлекательными для продвинутых пользователей, но одновременно повышает риск. В таких решениях грань между полезной оптимизацией и чрезмерным вмешательством в систему становится особенно тонкой.

Для наглядного сопоставления основных свойств рассмотренных утилит можно использовать следующие критерии: глубина вмешательства, удобство интерфейса, ориентация на приватность, ориентация на производительность, возможность отката и целевая аудитория.

Критерий	Winaero Tweaker	Ultimate Windows Tweaker	Hone	Win10Tweaker
Количество твиков	Сотни, с акцентом на настройку системы и	интерфейса Более 200, сбалансир	ованный набор Среднее число,	упор на готовые сценарии Сотни, включая агрессивные изменения
Приватность	Хорошее покрытие базовых механизмов теле	метрии Высокий акцент на р	rivacy-настройках Средняя, как	часть общей оптимизации Высокая, но с повышенным риском побочных эффектов
Производительность	Точечная ручная оптимизация	Хороший набор систе	мных параметров Сильный упор	на gaming-сценарии Высокий потенциал за счёт глубокой очистки
Кастомизация UI	Очень высокая	Высокая	Низкая	Средняя
Простота использования	Высокая для подготовленного пользователя	Высокая	Очень высокая	Средняя
Откат изменений	Частичный или по отдельным настройкам	Предусмотрен	Предусмотрен	Обычно присутствует, но зависит от сценария
Целевая аудитория	Энтузиасты и power users	Широкий круг пользо	вателей Геймеры	Опытные пользователи

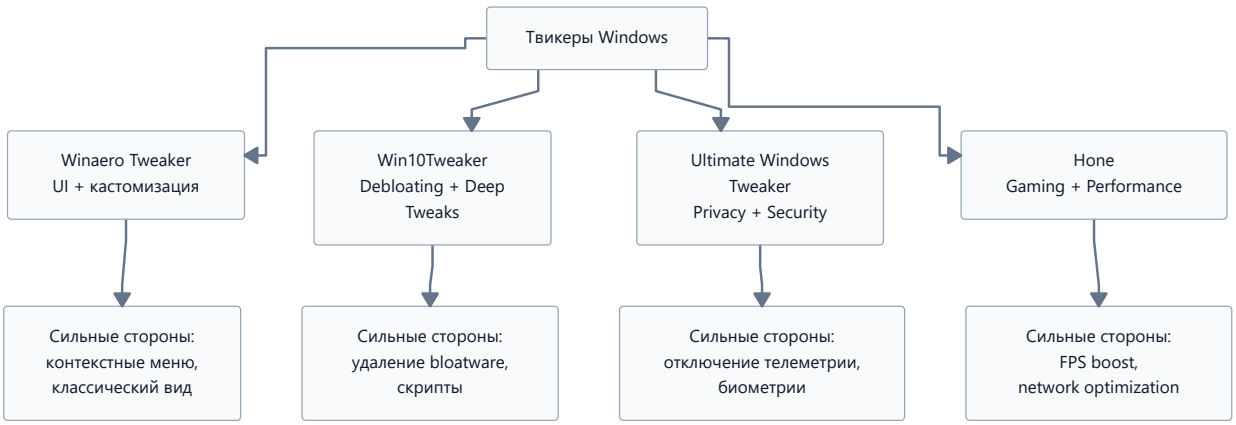

Рисунок 13. Сравнение функциональности твикеров

Сравнение показывает, что ни один из рассмотренных инструментов не закрывает все задачи одинаково хорошо. Одни решения сильны в кастомизации, другие — в приватности, третьи — в производительности. При этом у большинства из них есть общие ограничения: не всегда прозрачная модель состояния твика, неодинаковая глубина обратимости, смещение в сторону одного сценария и недостаточная связка между диагностикой системы и применением изменений.

Эта совокупность ограничений создаёт основание для разработки собственного программного средства. Перспективной выглядит модель, где одно приложение объединяет мониторинг состояния системы, категоризированные твики, безопасное применение, понятные описания рисков, резервное копирование и вспомогательные инструменты. Такая модель сохраняет достоинства рассмотренных решений [3] [4] [5] [10] и при этом снижает влияние их типичных недостатков: разрозненности, чрезмерной агрессивности и слабой прозрачности для пользователя.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА ДЛЯ КОМПЛЕКСНОЙ ОПТИМИЗАЦИИ WINDOWS

2.1. Постановка задачи и требования к программному средству

Анализ первой главы показывает, что оптимизация Windows редко сводится к одной настройке. В реальной работе проблемы идут рядом: лишние фоновые функции, нестабильный отклик интерфейса, разросшаяся автозагрузка, неочевидные сетевые и системные параметры, слабая наглядность при оценке текущего состояния компьютера. Рассмотренные твикеры — Winaero Tweaker, Ultimate Windows Tweaker, Hone и Win10Tweaker [3] [4] [5] [10] — чаще делают ставку либо на массовую модификацию системы, либо на отдельную группу настроек. Пользователь при этом не всегда получает цельный и управляемый сценарий. Поэтому в проекте Winsentials акцент сделан на связке мониторинга, безопасного применения твиков и сервисных инструментов внутри одного настольного приложения.

Во второй главе обосновывается структура Winsentials как единой среды для комплексной оптимизации Windows 10 и Windows 11. Под комплексной оптимизацией здесь понимается не только изменение отдельных параметров реестра или конфигурации загрузки, но и удобный пользовательский процесс: просмотр системных метрик, выбор подходящего твика, оценка риска, применение изменения, откат при необходимости, а также контроль вспомогательных сущностей, например элементов автозапуска и резервных копий. Поэтому проектируемое средство должно не просто выполнять низкоуровневые операции, но и делать их управляемыми, прозрачными и воспроизводимыми.

Функциональные требования к системе можно описать через основные пользовательские действия. Сначала приложение должно показывать состояние компьютера: процессор, оперативную память, графическую подсистему, накопители и сетевые адаптеры. Затем пользователь должен получить доступ к твикам, сгруппированным по понятным категориям. Для каждой настройки нужны назначение, рекомендуемое значение, уровень риска и указание на дополнительное действие после применения. Отдельное требование — обратимость: пользователь должен иметь возможность вернуть один твик в исходное состояние или восстановить ранее сохранённый набор параметров из резервной копии. Кроме того, в приложение включаются практические инструменты обслуживания Windows, прежде всего управление автозагрузкой и хранение снимков конфигурации.

Нефункциональные требования напрямую повлияли на архитектуру. Первое из них — безопасность применения твиков. Операция изменения системы должна быть ограничена конкретной настройкой, а не скрываться

внутри набора непрозрачных скриптов. Для этого нужны типизированные метаданные, формальный интерфейс применения и отдельный механизм чтения текущего состояния параметра. Второе требование — расширяемость: новые категории твиков, страницы интерфейса и фоновые сервисы должны добавляться без переписывания всей архитектуры. Третье — отзывчивость интерфейса, поскольку пользователь работает не только с настройками, но и с постоянно обновляемыми системными метриками. Четвёртое требование связано с совместимостью Windows 10 и Windows 11: конкретный твик может зависеть от сборки ОС, быть недоступным или требовать предупреждения.

Важным требованием стала ориентация на пользователя без глубоких навыков системного администрирования. Интерфейс не должен сводиться к техническим идентификаторам реестра, параметрам командной строки и внутренним системным ключам. Каждый элемент управления нужно описывать через ожидаемый результат: ускорение отклика, уменьшение фоновой активности, оптимизацию ввода, снижение лишней сетевой нагрузки и тому подобное. Для этого в модели данных каждого твика фиксируются краткое и подробное описание, рекомендуемое значение, тип элемента управления и последствия применения, включая необходимость выхода из системы, перезапуска компьютера, приложения или устройства. Это снижает порог входа и уменьшает вероятность ошибочного использования программы.

Отдельного внимания потребовало требование к целостности пользовательского сценария. В рамках Winsentials пользователь должен иметь возможность последовательно пройти весь путь от диагностики к применению изменения. Сначала он открывает обзорное представление и получает данные о ресурсах системы, затем переходит в нужную категорию твиков, выбирает конкретную настройку, оценивает описание и риск, применяет изменение, после чего при необходимости выполняет рекомендованное дополнительное действие. Если результат его не устраивает, он может сбросить отдельный твик или восстановить состояние из созданной ранее резервной копии. Поэтому система проектируется не как разрозненный набор страниц, а как единая среда поддержки решений при оптимизации Windows.

С учётом сказанного итоговая постановка задачи может быть сформулирована следующим образом: необходимо разработать настольное программное средство для Windows, обеспечивающее централизованное отображение системных метрик, безопасное применение и откат твиков, а также вспомогательные операции по обслуживанию конфигурации компьютера. Приложение должно совмещать понятный пользовательский интерфейс и низкоуровневую точность выполнения системных действий, при этом каждая операция обязана

быть формализована, проверяема и пригодна для повторного воспроизведения. Эти требования определили выбор архитектурного подхода и используемого технологического стека.

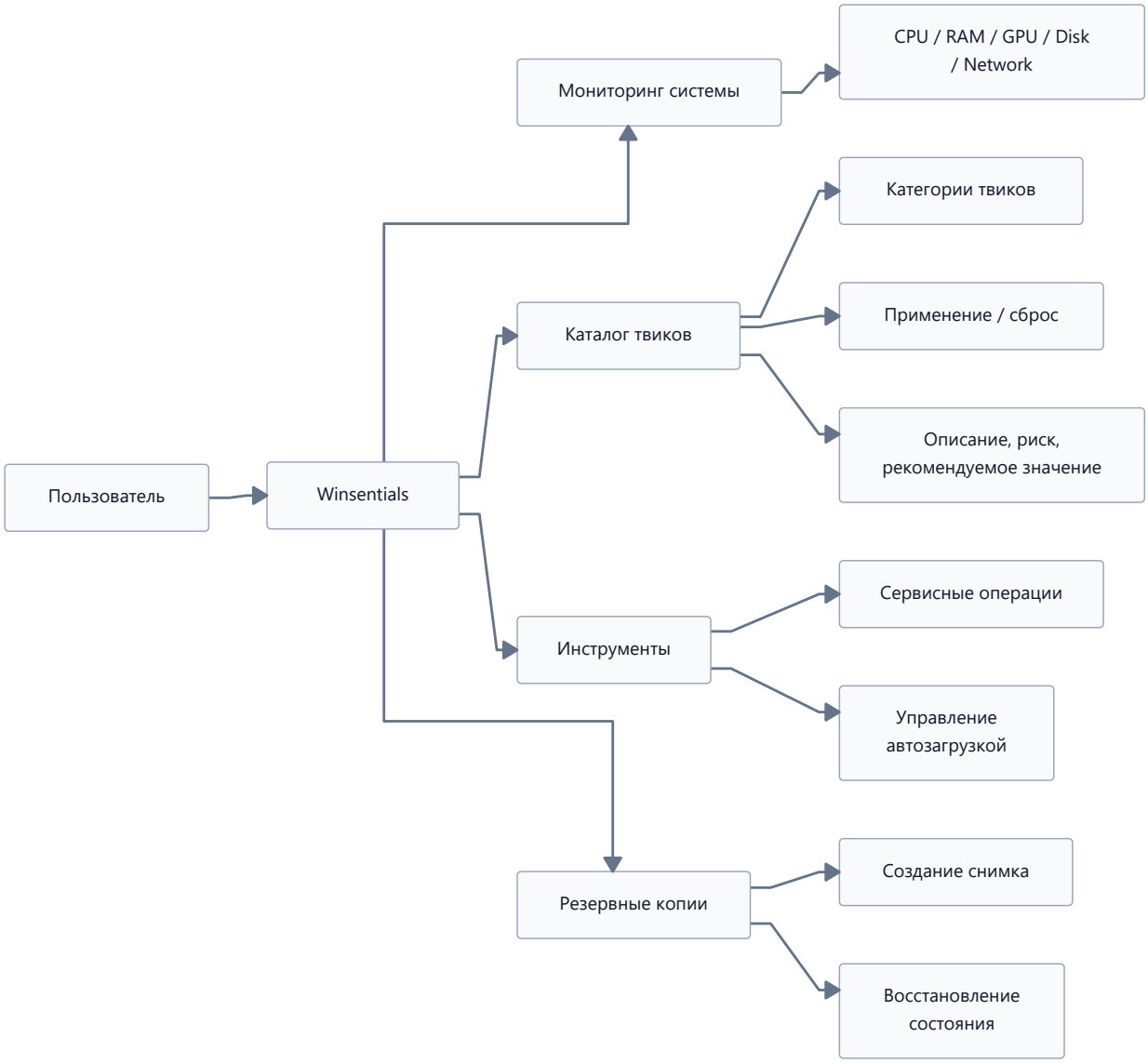

Рисунок 14. Функциональная структура программного средства Winsentials

На рисунке 15 показана главная страница Winsentials. Она используется как начальная точка работы с приложением: пользователь видит сведения о системе, состояние применённых твиков и основные показатели аппаратных ресурсов.

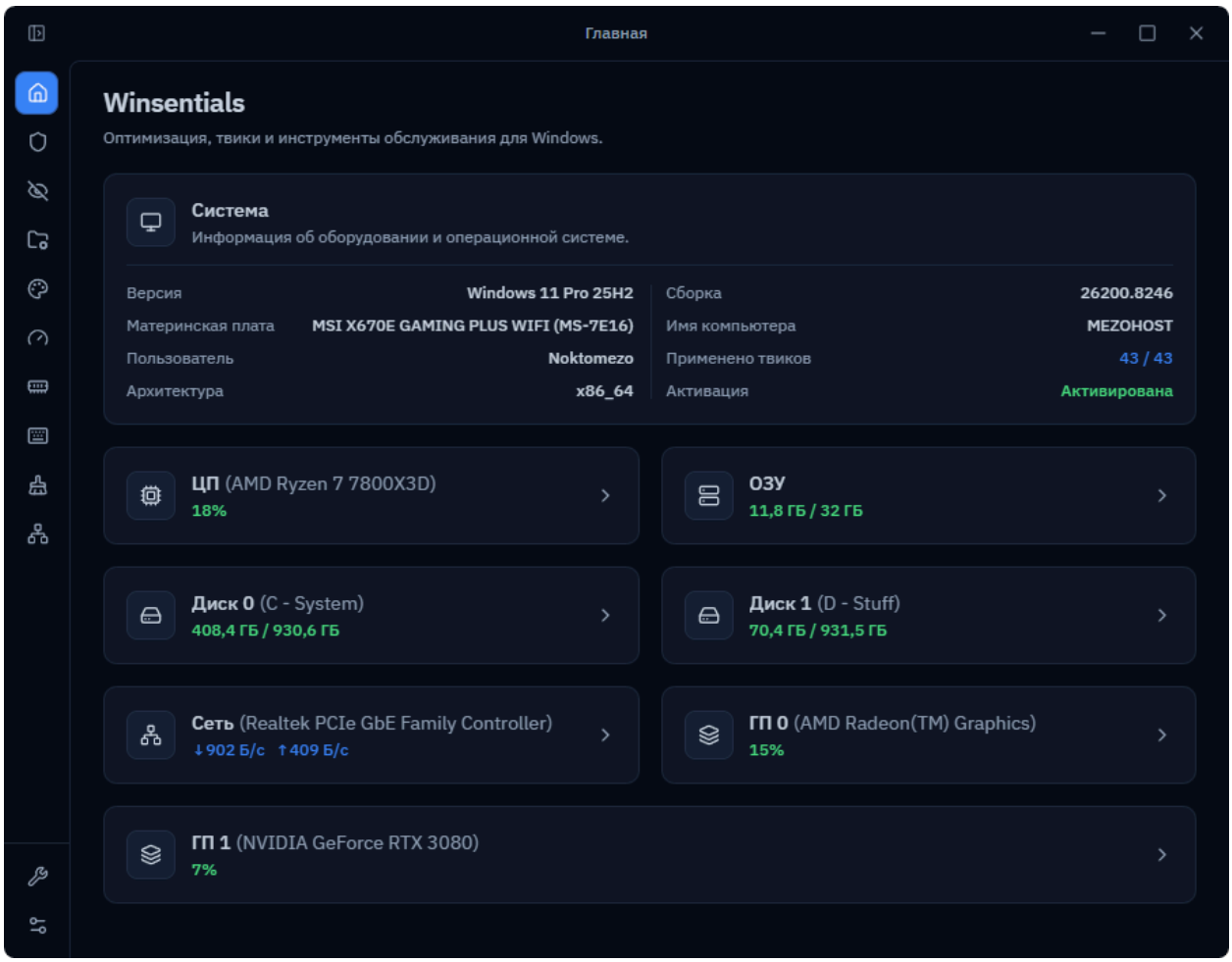

Рисунок 15. Главная страница программного средства Winsentials

2.2. Архитектура программного средства и выбор технологий реализации

В качестве архитектурной основы для Winsentials была выбрана настольная модель, в которой пользовательский интерфейс реализуется с применением веб-технологий, а системная логика исполняется в нативном слое. На практике это выражено использованием платформы Tauri v2. Данный выбор обусловлен несколькими факторами. Во-первых, Tauri даёт возможность создавать нативные приложения для Windows на сравнительно лёгкой оболочке WebView, что уменьшает расход памяти и размер итоговой поставки по сравнению с тяжёлыми браузерными контейнерами. Во-вторых, Tauri формирует безопасный мост между интерфейсом и системными командами: вызвать можно только функции, явно зарегистрированные разработчиком. В-третьих, связка Tauri и Rust [11] [12] [19] хорошо подходит для слоя, который изменяет системные параметры и работает с реестром, службами и конфигурацией загрузки Windows.

Архитектура приложения разделена на два контура. Первый контур — клиентская часть на React 19 и TypeScript в строгом режиме типизации. Этот

слой отвечает за маршрутизацию, отображение данных, локальное состояние интерфейса, локализацию и взаимодействие с пользователем. TanStack Router задаёт структуру экранов: обзор системы, категории твиков, настройки, резервные копии, инструменты автозапуска и детальные страницы ресурсов. Пользовательские предпочтения хранятся в Zustand и сохраняются через Tauri store, поэтому после перезапуска остаются тема оформления, материал окна, язык интерфейса и другие параметры. Визуальный слой построен на TailwindCSS v4 и компонентах shadcn/ui. Такая связка React, TypeScript, Zustand и TanStack Router [13] [14] [15] [16] [17] даёт модульную структуру без избыточной инфраструктуры.

Второй контур образует серверная, или системная, часть приложения, написанная на Rust. Этот слой инкапсулирует реальные действия над системой и выполняет роль контролируемого исполняемого ядра программы. Именно здесь находятся команды Tauri, доступные клиентскому интерфейсу: получение информации о системе, применение и сброс твиков, работа с резервными копиями, управление автозагрузкой, установка материала окна, а также выполнение сервисных действий наподобие выхода из сеанса и перезапуска компьютера. Регистрация команд Tauri [11] [12] [19] в одном месте создаёт чёткую границу между интерфейсом и системными изменениями: React-компоненты не работают с реестром и службами напрямую, а могут обращаться только к строго определённому набору функций.

В Rust-части выделяются несколько подсистем. Первая — подсистема твиков, построенная вокруг абстракции Tweak. У каждого твика есть идентификатор, метаданные и набор операций: применение, сброс, чтение текущего состояния и, если нужно, дополнительное действие. Отдельные реализации собираются в общий реестр твиков и реестр по категориям. Новая настройка добавляется как самостоятельный модуль, а во время работы становится доступной и командному слою, и пользовательскому интерфейсу. Вторая подсистема отвечает за мониторинг. В ней используется библиотека sysinfo [18] и, для Windows, счётчики производительности PDH. При запуске приложения создаётся фоновый поток, который регулярно обновляет сведения о процессоре, памяти, сети, графике и дисках. Благодаря этому интерфейс быстрее получает метрики и показывает состояние системы в реальном времени.

Отдельную роль играет модуль резервного копирования. При старте программы автоматически создаётся исходный снимок состояния, если таковой ещё не существует. Далее пользователь может формировать собственные резервные копии, просматривать их список, переименовывать, удалять и восстанавливать. На уровне реализации резервная копия представляет собой JSON-

снимок значений твиков, сохраняемый в пользовательском профиле. Важно, что восстановление выполняется не произвольной перезаписью системных данных, а повторным применением сохранённых значений через тот же типизированный механизм [19], что используется в обычной работе с твиками. Тем самым достигается единообразие логики и сокращается вероятность расхождения между «ручным» применением параметра и восстановлением из снимка.

Помимо основного механизма твиков, архитектура включает специализированные инструменты. Один из них — средство управления автозагрузкой, работающее с несколькими источниками: разделами реестра, папкой Startup и запланированными задачами. Такое решение особенно важно с практической точки зрения, поскольку вопросы оптимизации Windows часто связаны не только с низкоуровневыми параметрами, но и с сокращением фоновой нагрузки при запуске системы. Вынесение этого функционала в отдельный инструментальный раздел не перегружает страницы твиков и при этом сохраняет целостность программного средства.

Выбор технологий на стороне клиентского интерфейса также имеет прикладное обоснование. React [13] удобен для построения динамического интерфейса с большим количеством карточек, выпадающих списков, статусов и форм обратной связи. TypeScript [14] снижает риск ошибок при обмене данными между страницами и командами Tauri. TanStack Router [16] используется для прозрачной навигации с детальными страницами ресурсов и инструментов. Zustand [15] применён как достаточно компактное, но гибкое средство хранения локальных предпочтений без лишней архитектурной тяжеловесности. Наконец, связка TailwindCSS и shadcn/ui [17] ускоряет построение единообразной дизайн-системы, что особенно важно в приложении, где пользователь регулярно взаимодействует с большим числом однотипных карточек и элементов управления.

Архитектура Winsentials остаётся многоуровневой, но достаточно компактной. Пользовательский интерфейс отвечает за представление и сценарии взаимодействия, системная часть на Rust — за выполнение критически важных действий, а Tauri — за контролируемый обмен между этими слоями. Такое разделение [11] [12] [19] одновременно поддерживает удобство, производительность, безопасность и расширяемость. В дальнейшем подобная архитектура упрощает добавление новых твиков и развитие дополнительных модулей без нарушения уже выстроенной структуры приложения.

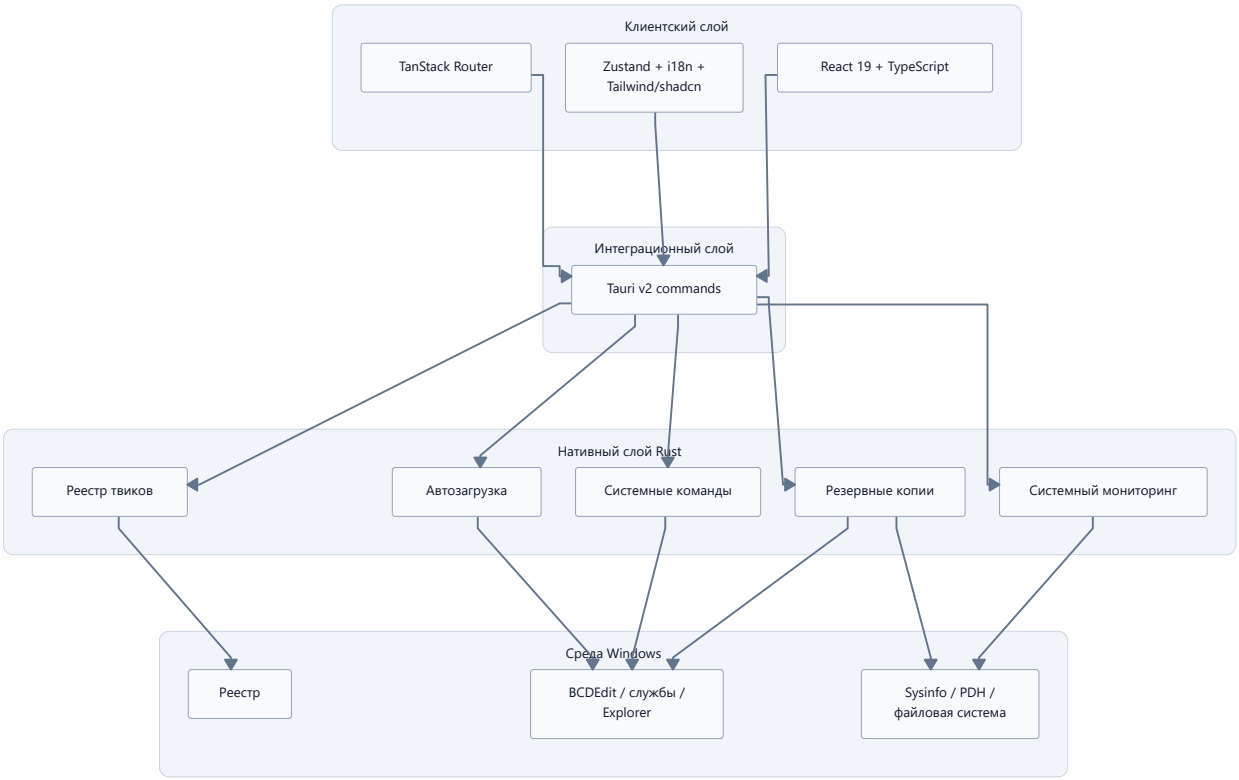

Рисунок 16. Архитектура взаимодействия интерфейса, команд Tauri и системных модулей

2.3. Классификация и описание реализуемых системных твиков

Реализованные в Winsentials твики организованы по категориальному принципу. Такая структура упрощает поиск нужной настройки и одновременно отражает предметную область, в которой выполняется оптимизация. В текущей версии программного средства основное наполнение сосредоточено в категориях оформления, поведения, ввода, сети, производительности и безопасности. При этом сама архитектура реестра твиков допускает дальнейшее расширение набора категорий без изменения базовых механизмов. Пользователь взаимодействует не с разрозненными скриптами, а с карточками настроек, каждая из которых имеет единый формат описания и единый цикл жизни.

Категория оформления включает твики, связанные с визуальным и поведенческим восприятием интерфейса Windows. Сюда относятся, например, возврат классического контекстного меню, ускорение предпросмотра миниатюр панели задач, изменение скорости мигания курсора, скрытие отдельных навигационных разделов проводника, а также устранение ярлычных стрелок и служебных суффиксов. Несмотря на кажущуюся вторичность таких настроек, в общей системе оптимизации они важны: с их помощью среду можно адапти-

ровать под привычный сценарий работы пользователя и снизить визуальный шум.

Категория поведения объединяет настройки, влияющие на логику работы системы и пользовательские сценарии. В эту группу входят, в частности, отключение задержки автозапуска приложений, открытие проводника сразу на разделе «Этот компьютер», отключение списков недавних элементов и частых мест, а также разблокировка скрытых параметров времени ожидания экрана блокировки. Данные твики повышают предсказуемость поведения системы и сокращают число лишних действий со стороны пользователя.

Категория ввода содержит настройки, влияющие на непосредственное взаимодействие пользователя с устройствами управления. В текущей реализации сюда входят ускоренный повтор клавиш и отключение акселерации мыши. Эти твики особенно востребованы в сценариях, где важна стабильность и воспроизводимость ввода: при наборе текста, работе в графических приложениях, а также в игровых задачах. Для подобных настроек важна не только возможность применения, но и простая интерпретация: пользователь должен видеть не техническое название параметра, а ожидаемый практический результат.

Категория сети охватывает изменения, связанные с сетевым стеком Windows. Здесь реализованы, например, отключение активного зондирования подключения, снятие ограничения пропускной способности QoS, включение алгоритма BBR2, а также включение сетевого offloading. Особенность данной группы состоит в том, что часть настроек способна заметно влиять на характер сетевого взаимодействия, но эффект зависит от конкретной инфраструктуры и оборудования. Поэтому архитектура метаданных твика предусматривает хранение уровня риска, поясняющего описания и, при необходимости, конфликта или ограничений по поддерживаемым версиям системы.

Категория производительности включает наиболее чувствительные с точки зрения результата настройки. В ней реализованы отключение Fault Tolerant Heap, отключение GameDVR, оптимизация MMCSS и настройка цепочки таймеров ядра. Эти твики ориентированы на снижение фоновых накладных расходов, стабилизацию времени кадра, уменьшение микрофризов и более предсказуемый отклик системы. В отличие от визуальных настроек, здесь особенно важны сопровождение пояснениями и контроль отката, поскольку пользователь должен понимать, что изменение затрагивает внутреннее поведение подсистем Windows.

Категория безопасности включает параметры, позволяющие снизить число навязчивых уведомлений и автоматических действий системы либо, наоборот, осознанно изменить защитное поведение Windows под конкретный

сценарий. В эту группу входят отключение автозапуска внешних носителей, предупреждений Центра безопасности, предупреждений при открытии файлов, а также изменение механизма UAC. Подобные твики нельзя рассматривать как безусловно полезные или вредные: их применение зависит от контекста использования компьютера. По этой причине для каждого такого элемента особенно важно явно фиксировать риск и отделять рекомендуемое значение от значения по умолчанию.

С инженерной точки зрения классификация важна не только для предметной области, но и для способа взаимодействия. В Winsentials твик может быть представлен переключателем, выпадающим списком или набором предустановок. Форма управления подбирается под природу параметра: простые бинарные изменения удобно применять через переключатель, а варианты с несколькими режимами — через селект или радио-выбор. Кроме того, каждая настройка содержит рекомендуемое значение, значение по умолчанию, текущее состояние и сведения о требуемом действии после применения. В результате карточка твика становится не просто кнопкой запуска, а законченной единицей управления конфигурацией.

Дополнительную ценность создаёт единая модель метаданных. Для каждого твика хранятся идентификатор, категория, название, краткое и подробное описание, тип управления, уровень риска, значение по умолчанию, рекомендуемое значение, а также сведения о конфликтующих сценариях и поддерживаемых сборках Windows. Такая формализация даёт клиентской части единые правила отображения карточек, а серверной — общий способ обработки операций статуса, применения и сброса. Поэтому классификация твиков в Winsentials не ограничивается навигацией; она напрямую связана с внутренней моделью данных и обеспечивает масштабируемость системы по мере роста числа настроек.

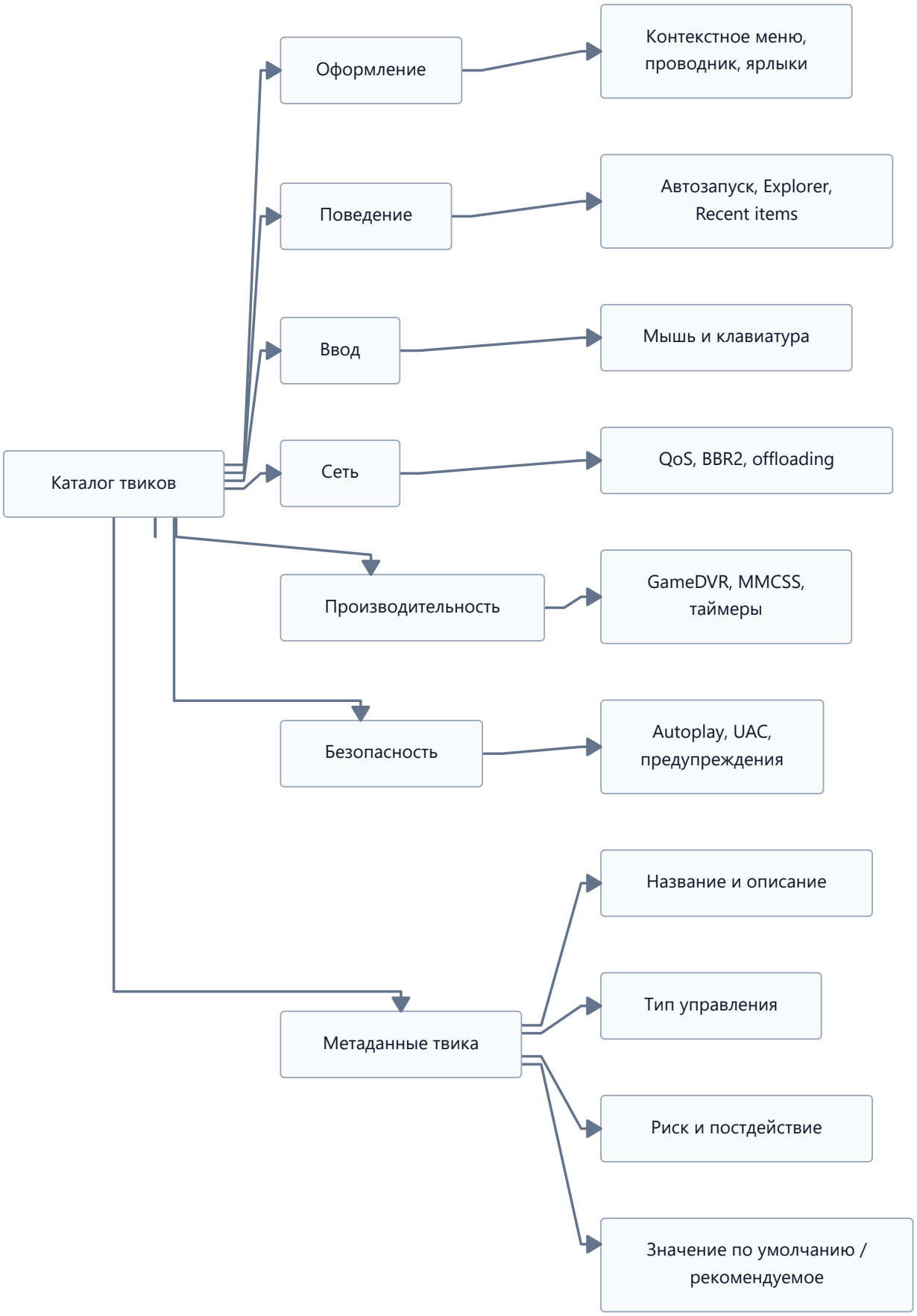

Рисунок 17. Классификация реализуемых твиков и их представление в пользовательском интерфейсе

2.4. Механизмы безопасного применения твиков и восстановления состояния системы

Одной из ключевых задач проектирования Winsentials стало обеспечение безопасного применения системных изменений. Для программного средства подобного класса недостаточно только записать значение в реестр или передать команду системной утилите. Нужен механизм, в котором каждая операция локализована, наблюдаема и обратима. В рассматриваемом приложении эта задача решается сочетанием типизированной модели твика, централизованного командного слоя, предварительного определения текущего состояния параметра и подсистемы резервного копирования.

Безопасность начинается уже на этапе описания твика. Каждая настройка реализуется как самостоятельный модуль с методами применения, сброса и получения фактического статуса. Разработчик не может добавить твик только с функцией «включить», не определив способ отката или проверки состояния. Такая дисциплина особенно важна в среде Windows, где одинаковый визуальный эффект иногда достигается несколькими низкоуровневыми способами. В Winsentials каждый твик оформляется как формальный контракт, а не как разовая операция. Благодаря этому пользовательский интерфейс может запросить текущее значение параметра до применения, после применения и после сброса.

Следующий уровень защиты — централизованный слой команд Tauri. Интерфейс обращается не к произвольным системным средствам, а к ограниченному набору зарегистрированных команд: получение списка твиков по категории, применение, сброс, чтение статуса, выполнение дополнительного действия и определение версии Windows. Такая схема минимизирует поверхность ошибок и упрощает контроль над исполнением. Клиентский код не содержит логики прямого изменения реестра, конфигурации загрузки или служб, что снижает риск расхождения между отображаемым состоянием и реальным состоянием системы.

После выполнения операции система повторно запрашивает статус твика и возвращает в интерфейс фактическое текущее значение. Это важное архитектурное решение, поскольку итоговое состояние не всегда должно определяться только тем параметром, который передал пользователь. Например, часть настроек может зависеть от сборки Windows, наличия конкретного компонента или сочетания нескольких системных значений. Повторное чтение статуса помогает не полагаться только на «ожидаемый» результат, а отображать пользователю подтверждённое состояние. Аналогичным образом работает и сброс: после возврата к исходным параметрам интерфейс получает фактическое

значение по умолчанию, а не абстрактный ответ об успешном завершении операции.

Важным элементом безопасного применения являются предупреждения и постусловия. Для отдельных твиков в метаданных фиксируется уровень риска и пояснение, а также необходимость выполнить дополнительное действие: выйти из системы, перезапустить компьютер, перезапустить приложение, службу или устройство. Наличие такого поля даёт интерфейсу возможность заранее предупредить пользователя о последствиях применения настройки. Это уменьшает число ситуаций, когда твик формально выполнен, но ожидаемый эффект не проявляется из-за отсутствия перезапуска. С инженерной точки зрения это также отделяет сам факт изменения параметра от связанных с ним эксплуатационных шагов.

Особую роль играет механизм резервного копирования. При первом запуске приложения формируется исходный снимок конфигурации, который служит опорной точкой для последующего восстановления. Дополнительно пользователь может создавать собственные резервные копии перед серией изменений. Каждая копия хранит набор значений твиков в JSON-формате и располагается в выделенном каталоге пользовательского профиля. Восстановление выполняется не прямой заменой файла конфигурации, а последовательным вызовом процедуры применения для каждого сохраненного твика. Так система использует тот же безопасный маршрут, что и при обычной работе: не обходит встроенные проверки и не дублирует бизнес-логику в нескольких местах.

С точки зрения надёжности важно и то, что процедура восстановления возвращает отчёт о результате, включая число успешно примененных значений и перечень тех твиков, для которых восстановление завершилось ошибкой. Такая модель предпочтительнее «молчаливого» отката, поскольку пользователь получает контролируемую обратную связь и может локализовать проблемные изменения. Аналогичная философия лежит и в основе инструментов автозапуска: операции включения, отключения и удаления записей выполняются явно, по известным типам источников, а не как недифференцированная очистка системы.

Ещё одним защитным механизмом служит учёт версии Windows. Для ряда настроек корректность применения зависит от конкретной сборки операционной системы. По этой причине приложение имеет отдельную команду получения номера сборки и ревизии, а модель твика может хранить ограничения по поддерживаемым версиям. Это предотвращает применение параметров в неподходящей среде и заранее показывает пользователю предупреждение

о недоступности настройки. В совокупности с метаданными риска такой механизм делает приложение более предсказуемым и снижает вероятность нештатного поведения системы после оптимизации.

Безопасность в Winsentials достигается не одним отдельным средством, а совокупностью проектных решений: строгой типизацией твиков, обязательным наличием операций статуса и сброса, централизованным слоем команд, отображением последствий применения, учётом версии Windows и встроенной подсистемой резервного копирования. Такая архитектура соответствует цели разработки программного средства для массового использования: пользователь может вносить полезные системные изменения без необходимости самостоятельно отслеживать все внутренние зависимости и риски. В итоге программное средство становится не просто «запускателем твиков», а управляемой средой конфигурирования Windows.

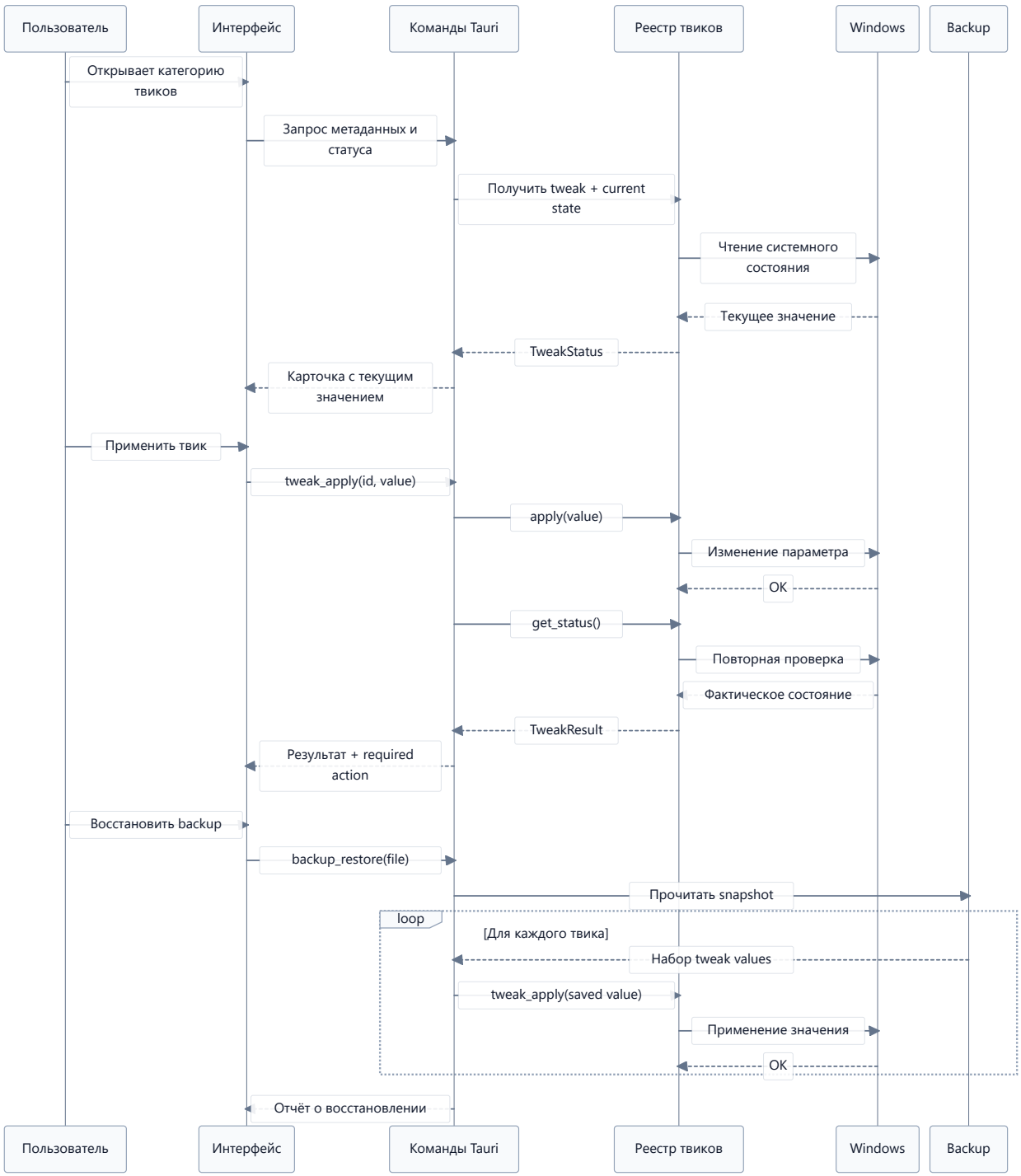

Рисунок 18. Последовательность безопасного применения твика и восстановления конфигурации

В ходе проектирования сформирована архитектура программного средства, которая сочетает компактный пользовательский интерфейс, нативный системный слой и формализованную модель работы с твиками. Такая структура создаёт основу для дальнейшей реализации и последующей оценки эффективности решений, что и будет рассмотрено в следующей главе.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Docs. Kernel-Mode Driver Architecture Design Guide [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/kernel/ (дата обращения: 02.11.2025).
2. Microsoft Docs. Windows kernel [Электронный ресурс]. – URL: https://learn.microsoft.com/en-us/windows-hardware/drivers/ddi/_kernel/ (дата обращения: 02.11.2025).
3. Winaero. Winaero Tweaker [Электронный ресурс]. – URL: https://winaero.com/winaero-tweaker/ (дата обращения: 02.11.2025).
4. The Windows Club. Ultimate Windows Tweaker 5 for Windows 11 [Электронный ресурс]. – URL: https://www.thewindowsclub.com/ultimate-windows-tweaker-5-for-windows-11 (дата обращения: 02.11.2025).
5. Hone. Optimize Your PC For Gaming [Электронный ресурс]. – URL: https://hone.gg (дата обращения: 02.11.2025).
6. InformatiWeb. Disable tracking systems (telemetry) of Windows 10 and 11 [Электронный ресурс]. – URL: https://us.informatiweb.net/tutorials/it/windows/windows-10-11-disable-windows-tracking-systems-telemetry--3.html (дата обращения: 02.11.2025).
7. Microsoft Support. Tips to improve PC performance in Windows [Электронный ресурс]. – URL: https://support.microsoft.com/en-us/windows/tips-to-improve-pc-performance-in-windows-b3b3ef5b-5953-fb6a-2528-4bbed82fba96 (дата обращения: 02.11.2025).
8. Wikipedia. Architecture of Windows NT [Электронный ресурс]. – URL: https://en.wikipedia.org/wiki/Architecture_of_Windows_NT (дата обращения: 02.11.2025).
9. FreshTechTips. How to Disable Telemetry in Windows 10 – Step-by-Step Guide [Электронный ресурс]. – URL: https://www.freshtechtips.com/2025/08/disable-windows-10-telemetry.html (дата обращения: 02.11.2025).
10. Dizzion. Windows OS Optimization Essentials Part 3 - Services & Scheduled Tasks [Электронный ресурс]. – URL: https://www.dizzion.com/post/windows-os-optimization-essentials-part-3---services-scheduled-tasks (дата обращения: 02.11.2025).
11. Tauri. Prerequisites [Электронный ресурс]. – URL: https://v2.tauri.app/start/prerequisites/ (дата обращения: 09.04.2026).

12. Tauri. Create a Project [Электронный ресурс]. – URL: https://v2.tauri.app/start/create-project/ (дата обращения: 09.04.2026).

13. React Docs. Managing State [Электронный ресурс]. – URL: https://react.dev/learn/managing-state (дата обращения: 09.04.2026).

14. TypeScript Docs. What is a tsconfig.json [Электронный ресурс]. – URL: https://www.typescriptlang.org/docs/handbook/tsconfig-json.html (дата обращения: 09.04.2026).

15. Zustand Docs. persist middleware [Электронный ресурс]. – URL: https://zustand.docs.pmnd.rs/reference/middlewares/persist (дата обращения: 09.04.2026).

16. TanStack Router Docs. Overview [Электронный ресурс]. – URL: https://tanstack.com/router/learn/docs/framework/react/overview (дата обращения: 09.04.2026).

17. Tailwind CSS. Get started with Tailwind CSS [Электронный ресурс]. – URL: https://tailwindcss.com/docs/installation/tailwind-cli (дата обращения: 09.04.2026).

18. docs.rs. sysinfo crate [Электронный ресурс]. – URL: https://docs.rs/sysinfo/ (дата обращения: 09.04.2026).

19. Klabnik S., Nichols C., Krycho C. The Rust Programming Language [Электронный ресурс]. – URL: https://doc.rust-lang.org/book/index.html (дата обращения: 09.04.2026).